

PANEL

Journal de lecture pour comics Marvel

Guide complet : explication du projet, installation, tutoriel de mise en place, gestion au quotidien, et comment connecter le catalogue à l'API officielle Marvel pour dépasser l'échantillon de départ.

Contenu du dossier fourni	
index.html	Le site (front), à héberger tel quel
config.js	Clés de connexion à ta base Supabase
supabase/schema.sql	Structure de la base + données de départ
scripts/import-marvel.js	Import du catalogue Marvel officiel (voir section 6)
README.md	Version texte de ce guide

1. Présentation & architecture

PANEL est une application web où chaque personne peut cataloguer sa lecture de comics Marvel : marquer un numéro comme lu, le noter, écrire un avis, et consulter son historique trié par série, par run ou par arc de lecture. L'application repose sur deux briques seulement, sans serveur à administrer soi-même.

Brique	Rôle	Où ça vit
Front (index.html)	Interface visuelle, logique d'affichage	Hébergé sur Vercel, gratuit
Supabase	Base de données Postgres, authentification, API générée automatiquement	Service géré, gratuit jusqu'à 500 Mo

Le navigateur de l'utilisateur parle directement à Supabase (pas de serveur intermédiaire à écrire). Deux tables structurent les données :

- **issues** — le catalogue de numéros. Partagé par tout le monde : tout compte connecté peut en ajouter, tout le monde peut les lire, y compris sans être connecté.
- **logs** — les lectures loguées (date, note, avis). Privée à chaque utilisateur grâce à la Row Level Security (RLS) de Postgres : une règle SQL garantit qu'on ne voit et ne modifie que ses propres entrées.

Chaque lecture crée une nouvelle ligne dans **logs** plutôt que d'écraser la précédente : relire un numéro ajoute une entrée de plus, ce qui donne un véritable historique chronologique (le "journal").

2. Ce qu'il faut installer

Aucun serveur Postgres à installer : Supabase l'héberge pour toi. Voici ce qu'il faut sur ta machine :

Outil	Pourquoi	Obligatoire
Git	Envoyer le code sur GitHub	Oui
Un éditeur de code (VS Code)	Modifier index.html / config.js	Oui
Compte GitHub	Héberger le code source	Oui
Compte Supabase	Base de données + authentification	Oui
Compte Vercel	Héberger le site en ligne	Oui
Node.js 18+	Tester en local, exécuter le script d'import Marvel	Optionnel
Client Postgres graphique (TablePlus, DBeaver)	Consulter la base sans passer par le site Supabase	Optionnel
Compte développeur Marvel	Récupérer le catalogue complet (section 6)	Optionnel

3. Tutoriel de mise en place

Étape 1 — Créer le projet Supabase

- Va sur supabase.com → "Start your project" → connecte-toi avec GitHub.
- "New project" : nom (ex. panel-comics), mot de passe de base à conserver, région proche de toi.
- Attends environ 2 minutes que le projet soit prêt.

Étape 2 — Charger le schéma de base

Dans Supabase : **SQL Editor** → **New query**. Colle tout le contenu de `supabase/schema.sql` et clique **Run**. Cela crée les tables `issues` et `logs`, leurs règles de sécurité, et environ 108 numéros de départ.

Étape 3 — Configurer l'authentification

Authentication → Providers → Email. Pour un test rapide en solo, tu peux décocher "Confirm email" (sinon chaque inscription exige de cliquer un lien reçu par mail).

Étape 4 — Connecter l'app à la base

Project Settings → API : copie **Project URL** et la clé **anon public**. Colle-les dans `config.js` :

```
window.PANEL_CONFIG = {
  SUPABASE_URL: "https://xxxxxxx.supabase.co",
  SUPABASE_ANON_KEY: "eyJhbGciOi...",
};
```

La clé anon est faite pour être publique : c'est la Row Level Security définie à l'étape 2 qui protège réellement les données, pas le secret de cette clé.

Étape 5 — Tester en local (optionnel)

```
npx serve .
# ou : python3 -m http.server 8080
```

Étape 6 — Mettre le code sur GitHub

```
cd comics-log
git init
git add .
git commit -m "Premier commit"
git remote add origin https://github.com/TON-COMPTE/panel-comics.git
git branch -M main
git push -u origin main
```

Étape 7 — Déployer sur Vercel

- Sur vercel.com, connecte-toi avec GitHub.
- "Add New... → Project", choisis le repo panel-comics.
- Laisse le Framework Preset sur "Other" (site statique, pas de build).
- "Deploy" : après ~30 secondes, une URL en `https://` est prête.

À chaque git push sur main, Vercel redéploie automatiquement.

Étape 8 — Nom de domaine personnel (optionnel)

Projet Vercel → Settings → Domains, ajoute un domaine acheté chez un registrar (Namecheap, OVH...) et suis les instructions DNS (en général un enregistrement CNAME). Environ 10 €/an, l'hébergement reste gratuit.

4. Gérer le projet au quotidien

Accéder et modifier le code

Il n'y a pas de serveur "back" à administrer : le back, ce sont le fichier supabase/schema.sql (structure) et le tableau de bord Supabase (données, règles, utilisateurs). Le front, c'est index.html et config.js, modifiables avec ton éditeur de code. Pour publier un changement :

```
git add .
git commit -m "Description du changement"
git push
```

Vercel redéploie automatiquement en quelques secondes.

Gérer la base de données

- **Table Editor** (Supabase) — voir et modifier les lignes comme dans un tableur, sans SQL.
- **SQL Editor** — requêtes ou modifications de structure plus poussées.
- **Client de bureau** (TablePlus, DBeaver) — dans Project Settings → Database, copie la "Connection string" et colle-la dans le client pour retrouver les tables comme dans n'importe quel Postgres.

Gérer les comptes utilisateurs

Authentication → Users dans Supabase : liste des comptes, possibilité de bloquer ou supprimer un utilisateur.

Sauvegarder les données

Le plan gratuit ne propose pas de sauvegardes automatiques historisées. Pour un export ponctuel : Database → Backups (plans payants), ou un export manuel via pg_dump avec la chaîne de connexion.

5. Coût réel de cette configuration

Service	Usage	Prix
Vercel (Hobby)	Hébergement du front	0 €
Supabase (Free)	BDD + Auth + API, jusqu'à 500 Mo	0 €
Nom de domaine (optionnel)	tonapp.com	~10 €/an

Le plan gratuit Supabase met le projet en pause après 7 jours sans requête ; une simple visite du site le réveille en quelques secondes. Pour un usage régulier par plusieurs personnes, le palier suivant est Supabase Pro à 25 \$/mois.

6. Aller plus loin : connecter l'API officielle Marvel

Le fichier `schema.sql` fourni au départ contient un échantillon saisi à la main (une douzaine de runs emblématiques, ~108 numéros) — pas le catalogue Marvel complet. Pour s'en approcher, Marvel propose une API développeur officielle.

Une précision importante avant de commencer

L'API Marvel donne des **métadonnées** (titres, numéros, dates, créateurs, miniature de couverture) — pas les pages scannées des comics. Ce n'est pas un accès aux planches complètes comme Marvel Unlimited : Marvel Unlimited est un service de lecture sous licence, fermé, sans API publique équivalente. Ce que tu peux construire ici, c'est un catalogue exhaustif à cataloguer et noter — comme Letterboxd le fait pour des films sans héberger les films eux-mêmes.

Pourquoi un appel direct depuis le site ne marche pas

- L'authentification Marvel exige une **clé privée** utilisée pour signer chaque requête (hash MD5 de l'horodatage + clé privée + clé publique). L'exposer dans le code du site la rendrait visible de tout le monde.
- L'API Marvel n'autorise pas les appels directement depuis un site tiers (CORS) — elle est conçue pour être appelée depuis un serveur.

La solution : un script qui tourne **en dehors du navigateur** (sur ta machine, ou plus tard via une tâche planifiée), qui interroge Marvel avec tes clés secrètes, puis écrit directement dans la table `issues` de Supabase. C'est le rôle du fichier `scripts/import-marvel.js` fourni avec ce projet.

Mise en place

- Crée un compte sur `developer.marvel.com`, section "My Account" : tu obtiens une clé publique et une clé privée.
- Dans Supabase : Project Settings → API → copie la clé **service_role** (différente de la clé anon : elle donne un accès complet à la base, à ne jamais mettre dans `config.js`).
- Copie `.env.example` vers `.env` à la racine du projet et remplis les quatre valeurs.
- Installe la dépendance puis lance un import ciblé :

```
npm install
node scripts/import-marvel.js --title "Amazing Spider-Man"
node scripts/import-marvel.js --series 1009610
```

Le script pagine automatiquement (100 résultats par appel, comme l'exige l'API Marvel) et insère chaque numéro dans `issues` sans doublon (upsert sur l'identifiant).

Limites à connaître avant de tout importer

- Le catalogue complet représente des dizaines de milliers de comics : un import total (`--all`) peut buter sur le quota d'appels quotidien de ton compte développeur Marvel — vérifie la limite actuelle sur `developer.marvel.com` avant de lancer.
- Marvel ne modélise pas la notion de "run" éditorial ni d'"arc de lecture" que cette app utilise : le script laisse ce champ vide pour les imports Marvel, à compléter manuellement si besoin.

- Les conditions d'utilisation de l'API Marvel (developer.marvel.com/terms) encadrent ce que tu as le droit d'afficher publiquement, notamment si le site devient accessible à d'autres personnes — à lire avant d'ouvrir l'app au-delà d'un usage personnel.

7. Dépannage courant

Symptôme	Cause probable / solution
Bandeau rouge "Configuration manquante"	config.js n'a pas été rempli avec tes vraies valeurs (étape 4).
"Email not confirmed" au login	Active/désactive la confirmation email (étape 3), ou clique le lien reçu par mail.
Une requête renvoie une liste vide	Vérifie les règles RLS dans Authentication → Politiques : policy manquante ou erreur dans auth.uid() = user_id.
"Failed to fetch" dans la console	Vérifie SUPABASE_URL : pas d'espace, pas de "/" final en trop.
Erreur 429 lors de l'import Marvel	Quota d'appels quotidien atteint : relance le lendemain, ou réduis la portée avec --title / --series.

Document généré pour accompagner le projet PANEL. Le code source complet (index.html, config.js, supabase/schema.sql, scripts/import-marvel.js) est fourni séparément.